Design layer

the whole layer, as one document

Contents

1 Hellhound 4

1.1 Foundation 4
1.2 Pending updates 5
1.3 System at a glance 7
1.3.1 Public modules 7
1.3.2 Whole-model relationships 8
1.4 Role and provider authority 8
1.4.1 Trust and configuration refinements 9
1.4.2 Metadata compatibility and signing keys 9
1.5 Login and identity 11
1.5.1 Transaction model and lifecycle 12
1.5.2 Verification and identity projections 13
1.6 Custody and refresh 14
1.6.1 Sealed records and the storage contract 14
1.6.2 Optional PostgreSQL integration 15
1.6.3 Session lifetime and installation 16
1.6.4 Refresh generation state and authority 17
1.6.5 Access operation and competing requests 19
1.7 Resource validation and provider operations 19
1.7.1 Local access-token validation 19
1.7.2 Userinfo and service token grants 20
1.7.3 Introspection and logout 20
1.8 Assertion signing and credential exchange 21
1.9 Transport and observations 23
1.9.1 Typed observations and correlation 24
1.10 Lifecycle and limits 25
1.10.1 Startup and cache replacement 25
1.10.2 Fetch ownership and cancellation 26
1.10.3 Public defaults and setters 26
1.10.4 Bounds and cleanup guarantees 27
1.11 Failures and caller decisions 27
1.12 Application responsibilities 28
1.12.1 Routes, origin and browser session 28
1.12.2 Identity and business permission 29
1.12.3 Custody and persistence 29
1.12.4 Refresh, retry and logout 29
1.12.5 Resource lifetime and observability 29
1.13 Testing and retained capability scope 29
1.13.1 Test and extension ports 30
1.13.2 Evidence and release conditions 31
1.13.3 Validation gates and continuous integration 31
1.13.4 Dependency provenance and release qualification 32
1.13.5 Benchmark method 33
1.13.6 Threat and authority coverage 33
1.13.7 Full capability disposition 34
1.14 End-to-end walkthrough 34

Glossary 34

Hellhound

1 Hellhound

1.1 Foundation

Hellhound verifies OIDC and OAuth evidence and owns user-token custody.

GOAL

Authenticate browser users with confirmed custody

A relying party completes authorization-code login over verified transport and receives checked identity only after confirmed session installation.

GOAL

Serve service and resource roles independently

A confidential service obtains its own access tokens without inventing a login redirect. A resource server validates access tokens without inventing client credentials.

GOAL

Retain the full protocol capability scope

Supported operations and deliberately unbuilt protocol families have explicit contracts and dispositions. External evidence constrains any growth or backend replacement.

NO-GOAL

Supply application permissions or browser sessions

Applications own business authorization, their authenticated session cookie, admission limits and protected resource requests.

NO-GOAL

Implement an authorization server or cryptography

The package targets Erlang. Authorization-server behavior, implicit, hybrid and password grants and JavaScript runtime support are excluded; cryptographic primitives belong to gose, kryptos and OTP.

INVARIANT

Only confirmed consumption authorizes code exchange

A rejected binding, rejected expiry admission, losing compare-and-set or unknown store outcome never authorizes exchange. A consumed authorization code is never resent.

mechanism

INVARIANT

Only fresh reservation authorizes refresh dispatch

Possible transmission quarantines the predecessor generation. Publication recovery never repeats provider refresh.

mechanism

INVARIANT

Checked identity and permissions have separate owners

Only Hellhound issues opaque identity, session and access-claim values. These values carry no application permission authority.

mechanism

INVARIANT

Provider observations cannot replace configured trust

Exact issuer, authentication method, algorithm policy and destination policy govern every provider operation. A token header or discovered endpoint cannot enlarge that authority.

mechanism

INVARIANT

Replacement caches cannot restore obsolete keys

Every cache incarnation discovers independently. Results from another incarnation cannot change its snapshot.

mechanism

INVARIANT

Ordinary diagnostics exclude credential material

Errors and typed observations omit codes, state, nonce, verifiers, credentials, tokens, raw claims and arbitrary foreign terms. VM inspection remains outside that guarantee.

mechanism

PRINCIPLE

Ask callers for decisions and retain owned facts

Configuration stays pure until validation. Protocol state and concurrency remain behind the public client, session, validator and store boundaries.

PRINCIPLE

Preserve uncertainty until authority resolves it

Refusal, proven non-action, possible effect and confirmed publication remain different outcomes. A convenient retry classification cannot erase them.

PRINCIPLE

Compose through caller-owned projections

Claim decoders preserve application types. Relay admission and Fabric approver policy compose through compiled consumer recipes without runtime dependencies.

1.2 Pending updates

build **Adopt later protocol families through explicit contracts** 2026-10-06

PAR, JAR, DPoP, JWT-bearer grant and dynamic registration remain retained intent. Each requires its own configuration, typed failures, public consumer and external suite before support is claimed.

ADR-0009

ruling **Choose optional security profiles and flow extensions** 2026-10-06

JARM and FAPI 2.0 security/message signing remain optional scope. Device grant, token exchange and back/front-channel logout remain explicit unsupported gaps. Social-provider quirks and broader refresh-scope profiles need a concrete growth decision.

ADR-0009

ruling **Define quarantine reconciliation authority** 2026-10-06

No public operator clears quarantine or turns uncertain reservation into dispatch. Define provider/generation reconciliation authority; the current caller action is reauthentication.

ADR-0005

ruling **Decide the required shutdown completion boundary** 2026-10-06

stop observes supervisor exit for at most five seconds and returns Nil even on expiry. Whether a consumer needs confirmed fetch and admitted-store-call completion before releasing captured resources is unresolved.

ADR-0011

verify **Verify deadline return latency during cleanup** 2026-10-06

`complete_login` propagates one admission deadline. A timed-out store worker is killed and its monitor may be awaited for up to one additional second before reply draining; synchronous telemetry also runs in the emitting process. Verify whether the promised whole-operation bound requires stronger completion semantics.

ADR-0011

verify **Verify third-party initiation and certification limits** 2026-10-06

Independent coverage must establish third-party initiation, RP logout's browser/session effects and unsigned-token refusal under the selected profiles. Certification requires the exact release's selected-suite acceptance and independent security review. Recorded regression receipts cannot confer certification.

ADR-0008

ruling **Choose kid-less rotation availability policy** 2026-10-06

A kid-less wrong-key signature can report bad signature without a key refresh. Refreshing on that failure needs an explicit bounded policy and attacker-load evidence.

ADR-0008

verify **Check claims carried over from retired guides against code** 2026-10-17

The public modules, public defaults and setters, application responsibilities, validation gates, dependency provenance and benchmark method sections state facts that earlier guides held. A grep of the source and a run of the gates has not yet confirmed every default value, setter name and responsibility clause.

verify **Retain the upstream location of the oidcc conformance harness** 2026-10-17

The oracle record pins the harness the differential corpus follows by revision and licence, and keeps no upstream location. Recover the location and record it beside the pin.

build **Add application-agnostic assertion signing and exchange** 2026-10-27

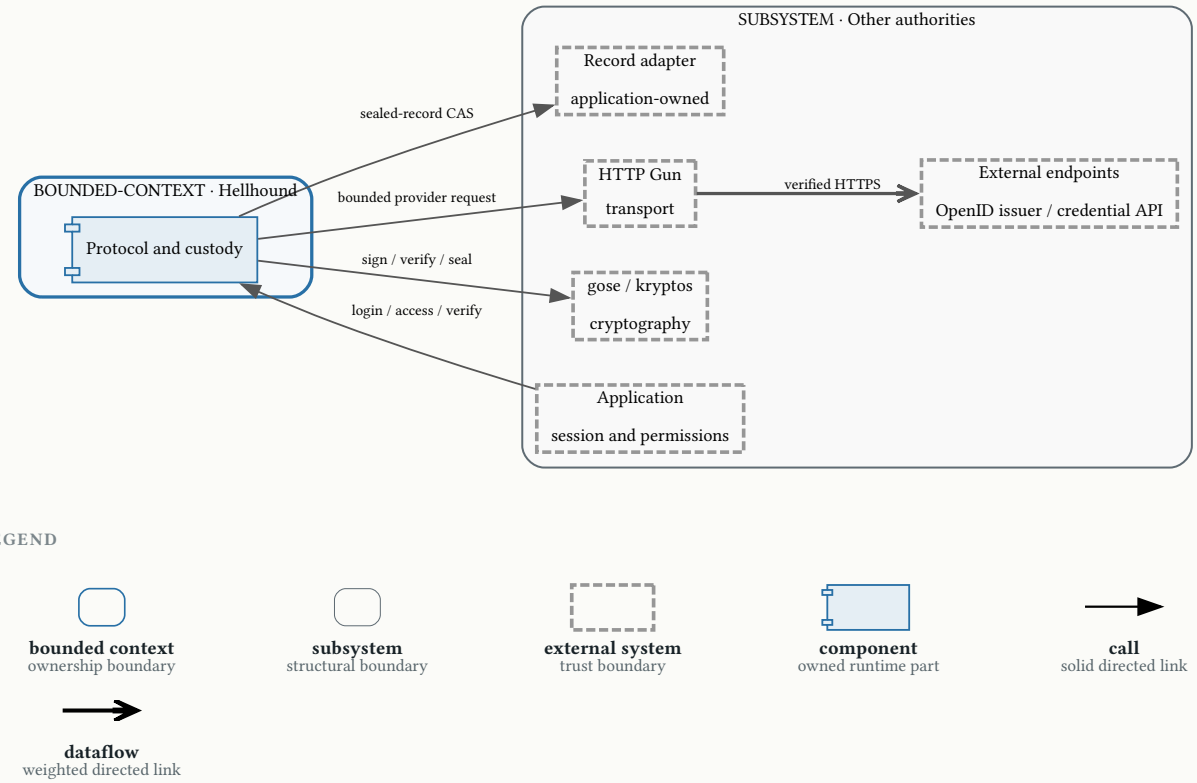
Accepted scope: expose signed-JWT creation and a generic HTTP token-exchange API. GitHub App installation credentials are a concrete consumer, while provider-specific paths, claims and response formats do not become GitHub-specific runtime APIs. Develop the capability through failing behavioral tests before implementation, then refactor with those tests passing. This scope does not itself establish support for RFC 7523 authorization grants or RFC 8693 token exchange; each protocol needs its own evidence.

ADR-0021

1.3 System at a glance

One Hellhound context owns protocol and custody; its neighbors retain distinct authorities.

VIEWPOINT CONTEXT-OWNERSHIP · ALTITUDE L1 · BOUNDARY Authentication and authorization owners



The Hellhound boundary owns checked evidence and token state. The application owns adapter correctness, connection lifetime and business decisions.

- A browser starts a `Login transaction` Hellhound binds the callback and installs `Custody`. A later request restores that custody and obtains a usable token.
- A service client performs client credentials or introspection without login. A resource validator uses the same issuer cache without user custody.
- Each `Client` owns its `Provider` cache, HTTP pool, sweeper and optional in-memory stores. A durable Store captures application-owned resources.

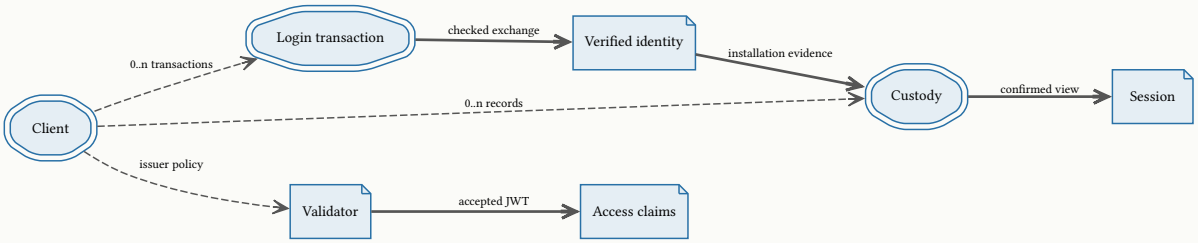
1.3.1 Public modules

Module	Purpose
`hellhound`	client lifecycle, login, sessions, access tokens, userinfo, client credentials, introspection and logout
`hellhound/config`	the opaque Config with `new`, `service_client`, `resource_server` and `with_*` setters
`hellhound/store`	the sealed-record storage port for durable sessions and logins
`hellhound/resource`	local JWT access-token validation under RFC 9068
`hellhound/assertion`, `hellhound/exchange`	standalone signed assertions and bounded credential exchanges
`hellhound/telemetry`	typed Sinal events with caller correlation
`hellhound/testing`	scripted local provider, test PKI, in-memory store and store conformance check

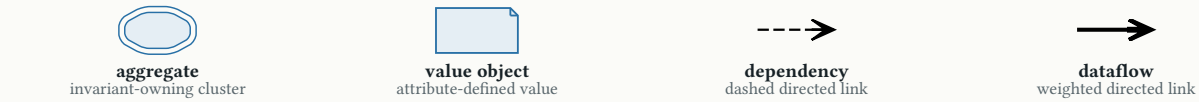
- `with_correlation` returns a pure client view whose events and HTTP Gun requests carry a Sinal correlation value. The HTTP Gun client is labelled `"hellhound"`.

1.3.2 Whole-model relationships

VIEWPOINT DOMAIN-MODEL · ALTITUDE L2 · CONTEXTS Authorities and checked projections



LEGEND



Identity is immutable authentication evidence; custody is mutable token authority. Access claims are an independent resource projection.

Model	Identity and authority	Canonical unit
Client, provider snapshot	Client names; snapshot accepted by one cache incarnation	Role and provider authority
Login transaction	state digest, version, expiry; one confirmed consumption	Login and identity
Custody record, refresh dispatch	reference digest, store version, token revision, dispatch and command ids	Custody and refresh
Session, recovery, access token	opaque owner-bound projections or retained commands	Custody and refresh
Validator, access claims	configured resource policy; checked token facts	Resource validation
Telemetry events	observed outcomes and correlation; no decision authority	Transport and observations

- The model has three mutable invariant owners: client/cache lifecycle, login transaction and custody record. Identity, session, token, validator and recovery values are immutable views or commands.
- Telemetry events are observed facts; they are neither durable custody events nor authorization commands. Correlation is observational and never changes admission.

1.4 Role and provider authority

Configuration admits a role before processes or provider effects exist.

Configuration admission

responsibility

Bind issuer, role, authentication, policy, limits and stores.

interface

config.new, `service_client` or `resource_server`; pure setters and validate; hellhound.new returns Client or StartError.

interactions

Validation derives internal settings, transport policy, sealing ring and stable names.

invariants

No connection or process starts during configuration. Invalid settings produce all detected ConfigErrors.

failure

Malformed issuer, redirect, scopes, trust, credentials or limits fail before startup.

Role	Required configuration	Admitted operations
Relying party	issuer, client id, redirect, authentication	login, custody, service grants, introspection and local validation
Service client	issuer, client id, confidential authentication	client credentials, introspection and validation; no login
Resource server	issuer; no client identity	local access-token validation; no grants or introspection

1.4.1 Trust and configuration refinements

- Issuer is an absolute HTTPS URI with no userinfo, query or fragment. Discovered issuer must equal it exactly. Redirects are absolute, fragment-free HTTPS except explicit loopback policy.
- A per-login redirect is chosen by exact string equality from the default and immutable allowlist. Near matches by port, case, slash or encoding fail before storage. Each address must be registered with the provider.
- Client authentication is PublicClient, secret basic, secret post, secret JWT or private-key JWT. The selected method never falls back silently; service clients require credentials.
- SigningAlgorithm admits RS256, PS256, ES256 or EdDSA, excluding none and HMAC. Keys must fit their algorithm, intended signing use and strength; RSA requires at least 2048 bits and unsupported curves are excluded.
- Secrets and signing/sealing keys are opaque redacted values. Durable stores require an admitted 32-byte sealing key; previous keys only open old records.
- Scope strings follow RFC 6749 bytes and preserve case. OIDC login adds openid; authorization options bound each value to 2 KiB and cannot override reserved parameters.
- Configuration includes response mode, issuer-parameter policy, prompt, max age, login hint, ACR, UI locales, destinations, trust, scopes, durations and store selection. Caller-owned option records extend `default_login` or `default_logout` by record update.

Duration or capacity	Admitted range	Default
provider request	1 ms–5 min	10 s
startup	1 ms–10 min	15 s
store and complete login	1 ms–10 min each	5 s / 30 s
pending lifetime / capacity	1 s–1 day / 1–10M records	10 min / 100K
session lifetime	positive idle ≤ absolute ≤ 1 year	1 h idle / 12 h absolute
clock tolerance	0–300 s	5 s
refresh margin / wait	0–1 h / 0–1 min	30 s / 5 s
provider body	1 KiB–64 MiB	1 MiB

1.4.2 Metadata compatibility and signing keys

Provider cache

responsibility

Authenticate discovery, admit compatible metadata and maintain signing keys.

interface

Cached metadata/keys, bounded first-attempt outcome and unknown-key refresh.

interactions

Native provider parsing uses HTTP Gun; native JOSE selects compatible keys after checking algorithm policy.

invariants

At most one active fetch per incarnation. Failed reload keeps its last accepted snapshot. Replacement starts empty.

failure

Unavailable cache is ProviderNotReady; incompatible metadata is typed refusal; malformed keys or metadata never become success.

- The discovery URI appends /.well-known/openid-configuration to the configured issuer. Discovery and initial JWKS share the startup deadline; background rediscovery permits two provider request timeouts.
- Cache-Control lifetime is clamped to 60 s–24 h, default 1 h. Failed discovery retries with backoff 1–60 s. Failed reload retains the live snapshot.

- An unknown kid can request keys once; a new kid refreshes immediately, repeated identifiers are throttled to once a second, and at most 64 identifiers are remembered. Multiple callers share active work.
- Unusable JWKs are skipped. X.509 members x5c, x5t, `x5t#S256` and x5u are stripped before gose parsing; key material is verified without certificate-chain trust. TLS trust anchors authenticate transport, not token-key pinning.
- Login requires provider code-flow, authentication, signing and response-mode compatibility. Requiring PAR or signed requests is unsupported. Default PKCE policy requires advertised S256.
- AssumeS256WhenUnadvertised accepts only omission, only for confidential clients. An explicit list without S256 still fails. The same verifier/challenge and nonce remain mandatory; the provider's enforcement cannot be inferred. ADR-0003

BEHAVIOR

Unavailable fresh discovery grants no token authority

Given A replacement cache has no accepted provider snapshot.

When The caller requests validation or login.

Then The operation reports provider unavailability until compatible fresh discovery succeeds.

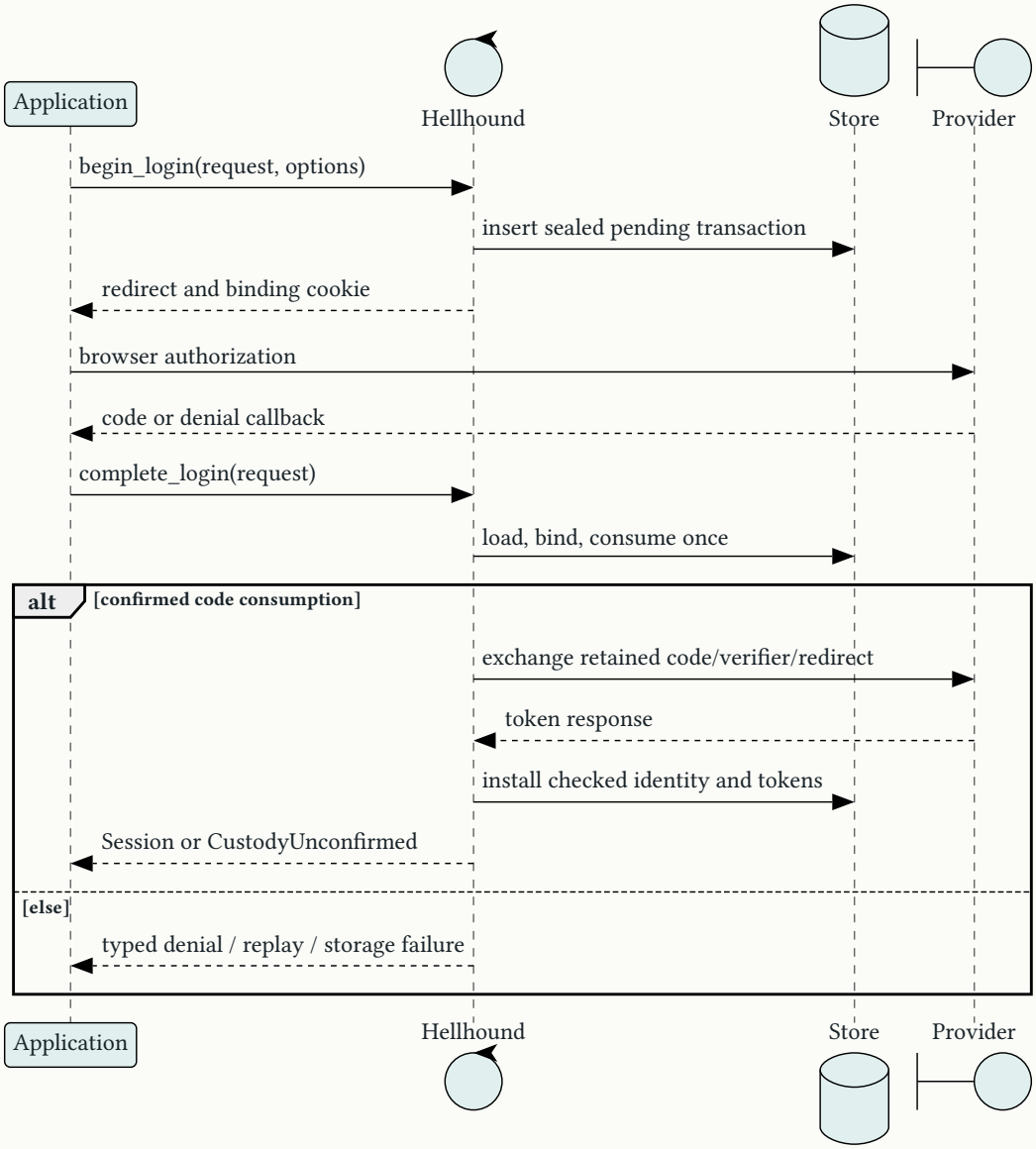
Then No earlier startup snapshot becomes authoritative.

level: boundary area: Provider readiness

1.5 Login and identity

A bound callback consumes one transaction before any code exchange.

SEQUENCE Code login and confirmed custody



Only confirmed consumption authorizes exchange. Installation recovery reuses its command and cannot re-enter the provider step.

Login boundary

responsibility

Generate independent login material, bind callbacks, consume once and issue checked identity/session.

interface

`begin_login`; `login_response`; `complete_login`; `recover_custody`; identity accessors and `decode_claims`.

interactions

Callback parser, transaction CAS port, native token client, JOSE verifier and custody installation.

invariants

Binding failure never consumes a legitimate login. Exchange never precedes confirmed consumption. Session construction requires custody confirmation.

failure

Malformed callback, binding refusal, expiry, replay, changed record, store failure, denial, exchange uncertainty, rejected identity and uncertain installation stay typed.

1.5.1 Transaction model and lifecycle

Login status vacant pending consumed expired

Login transaction aggregate stateful OIDC login owned by Hellhound

One pending authorization-code exchange and its bounded terminal evidence.

ATTRIBUTES

derived **Status** · Login status login-status · login-lifecycle
Derived from sealed record, store version and expiry at decision time.

derived **Binding material** · state, nonce, verifier, redirect, browser digest
Fresh cryptographic material and exact selected redirect retained at creation.

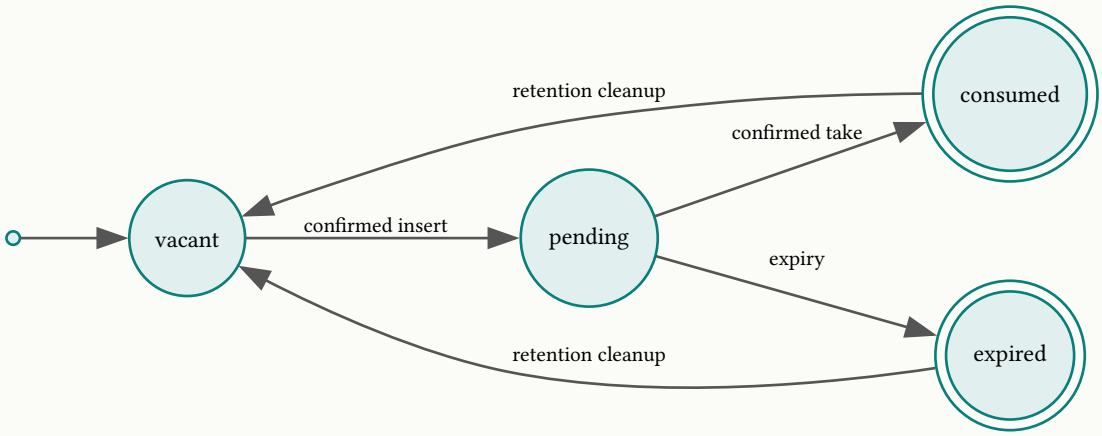
derived **Lifetime and version** · wall-clock seconds and store version
Creation/expiry and CAS concurrency evidence. The record remains until expiry plus one login lifetime.

RELATIONSHIPS

`n : 1` Belongs to one configured client and issuer.

`1 : 0..1` A confirmed code exchange may produce one checked identity installation.

STATE MACHINE Single-use transaction



Consumption withdraws exchange permission permanently. A terminal record is retained to distinguish replay from missing state.

- State, nonce and verifier each use 32 independent CSPRNG bytes. Their base64url representation has 43 characters; S256 derives the challenge from the same verifier.
- `begin_login` reads a shaped existing binding cookie or creates one. Tabs reuse that browser binding but receive independent state/nonce/verifier material. Transactions retain the binding digest, never the cookie value.
- `login_response` produces 303, Location and no-store and sets `__Host-hellhound_binding` with Secure, HttpOnly, Path=/, no Domain, and SameSite Lax for query or None for form post. Cookie lifetime exceeds login lifetime by five minutes.
- Callbacks require GET query or form POST of application/x-www-form-urlencoded according to configured mode. Parsing rejects duplicate/ambiguous parameters, malformed encoding, missing state/code, empty code and oversized input: 16 KiB total, 4 KiB per value.
- State and browser binding compare in constant time. A present iss equals the configured issuer; absence follows the explicit issuer-parameter policy and advertised requirement. Invalid binding performs no consumption or exchange.

- Consumption writes a sealed spent tombstone by CAS at the version read. Losing callbacks reload terminal disposition. The early clock check is followed by AtVersionBefore using the material validity deadline. The store checks version first and samples its wall clock under exclusive write authority after waiting for that authority. Equality refuses consumption; unknown writes send nothing. The deadline bounds that protected decision, not commit acknowledgement, code exchange or response completion. ADR-0013

BEHAVIOR

A bound denial consumes without exchange

Given A pending transaction belongs to the callback's browser and issuer.

When The caller completes a provider denial.

Then The transaction becomes consumed if its consumption is confirmed.

Then The caller receives the normalized denial and no code exchange occurs.

level: boundary area: Login consumption

BEHAVIOR

A queued login cannot consume after its deadline

Given A valid bound callback waits for storage write authority.

When Its validity deadline is reached before the protected decision.

Then Consumption is refused as expired and no provider code exchange occurs.

level: boundary area: Login consumption

BEHAVIOR

Timely consumption can finish after its deadline

Given Consumption was decided before the validity deadline.

When The write acknowledgement arrives after that deadline.

Then A confirmed write can authorize exchange subject to the remaining operation budget.

Then An uncertain write never authorizes exchange.

level: boundary area: Login consumption

BEHAVIOR

A losing callback cannot repeat exchange

Given One callback already consumed a transaction.

When Another callback completes the same transaction.

Then It reports replay and issues no identity or provider exchange.

level: boundary area: Login consumption

1.5.2 Verification and identity projections

- The code request retains configured client authentication, exact redirect, verifier and nonce. A consumed code is not restored after rejection, timeout, rejected identity or possible send. Proven no-send also requires a new login.
- Verification first rejects unsupported compact/JWE shape, none or non-allowlisted algorithm. It selects algorithm-compatible keys and delegates signature verification to gose; Hellhound performs its stricter claim profile.
- ID token is mandatory for login. Issuer is exact; audience is exactly the client; a present azp matches it; sub, iat and exp are required. Expiry is strict; iat, nbf and max-age authentication checks admit configured tolerance.
- Nonce is mandatory and equals the transaction. A present `at_hash` is checked against access material. Mistyped azp/`at_hash` are rejected; `max_age` requires a nonfuture sufficiently recent `auth_time`.
- VerifiedIdentity stores authenticated issuer/subject and checked claims. Email is optional and is never a stable identity key. Standard profile fields, `email_verified`, `auth_time`, ACR, AMR and caller-decoded claims retain absent values honestly.

- `decode_claims` accepts the caller's ordinary JSON Decoder and returns caller-owned values or DecodeErrors. Decoding a permission claim does not itself authorize a business action.
- One monotonic login deadline is passed through stores, exchange, key refresh and installation. Before exchange expiry is LoginTimedOut; uncertain installation is CustodyUnconfirmed. Cleanup and synchronous observation limits are stated in Lifecycle and limits.

BEHAVIOR

Uncertain installation recovers without exchange

Given Identity was checked and installation acknowledgement is uncertain.

When The caller submits the retained custody recovery to its client.

Then Custody repeats the same installation command and returns its existing confirmed session when known.

Then No authorization code is exchanged again.

level: boundary area: Login installation

1.6 Custody and refresh

Sealed CAS records retain token-generation authority across processes and nodes.

Custody

responsibility

Own verified session material, expiry, token revision, refresh state and bounded command receipts.

interface

`restore_session`, `access_token`, refresh, `recover_refresh`, logout; Store is an extension port.

interactions

Record sealing, CAS transitions, wall clock and native provider calls admitted by dispatch.

invariants

One fresh refresh dispatch per generation. Publication replaces access and refresh material together. Tombstones prevent late installation resurrection.

failure

Missing, lost, unreadable and unavailable custody differ; uncertainty is never permission to resend.

1.6.1 Sealed records and the storage contract

Sealed-record compare-and-set port

contract

Let applications persist records without owning OAuth transitions.

responsibility

Store key, version, `expires_at` and opaque sealed bytes; Hellhound owns interpretation.

interface

get(key); put(record, WriteCondition); `delete_expired`(now).

interactions

Captured adapter closures may use a caller-owned database pool. The same port can hold prefixed login and session rows.

invariants

Absent inserts only an absent key. AtVersion replaces only the expected version. AtVersionBefore additionally requires a future validity deadline at the protected decision. Written confirms the write; VersionConflict and DeadlineElapsed confirm no write. Version conflict takes precedence over expiry. Delete includes equality at retention expiry.

failure

StoreUnavailable proves non-action, StoreFull proves refusal, StoreOutcomeUnknown preserves a possibly completed write.

- The Write condition is a closed choice: Absent, AtVersion(version), or AtVersionBefore(version, deadline). No guarded insertion or arbitrary predicate is implied. Missing rows fail either versioned condition with VersionConflict. Only the guarded condition may return DeadlineElapsed. Each confirmed result is a Write outcome and AtVersionBefore carries its Validity deadline.
- The adapter owns the decision clock: the built-in actor uses its configured Unix-second wall clock; a durable adapter documents its synchronized wall clock and precision. Sample only after exclusive write authority is acquired. Retention and validity are independent. Captured pools remain application-owned.
- Adapters must implement the complete contract without a weaker fallback. Conditional refusals preserve bytes, version and retention. A lost acknowledgement remains StoreOutcomeUnknown; ordinary unguarded writes remain available for valid late publication. ADR-0013

BEHAVIOR

A conditional refusal leaves the record unchanged

Given A write requires absence, an expected version or an expected version before a deadline.

When The protected decision finds the condition unsatisfied.

Then The store returns the specific refusal and preserves the record.

Then A version conflict takes precedence over deadline expiry.

level: boundary area: Conditional storage

- Keys are login: or session: plus SHA-256 digest of state/reference. Plain bearer references, verifiers and tokens never reach the adapter. Store version changes on every accepted record mutation; token revision changes only on publication.
- AES-256-GCM uses a fresh 96-bit nonce; format is version byte, eight-byte key digest id, twelve-byte nonce, sixteen-byte tag and ciphertext. Additional data binds record kind, key and version.
- Current key seals; current and previous keys open. An unreadable or tampered record fails closed. GCM nonce/call budget requires rotation well before about 2^32 seals per key.
- AEAD prevents forgery or moving bytes to another key/version. It does not prevent restoration of a whole older row under its original key/version; applications protect writers, backups and rollback authority.
- In-memory stores use an ephemeral key and epoch-bearing reference. Restart can yield SessionLost; a missing durable reference yields SessionNotFound. A durable store's lifecycle and capacity remain application-owned.
- Every port call executes in an isolated monitored worker and catches adapter exceptions. Timeout/crash maps reads to unavailable and writes to unknown; termination drains the late private reply. Killing that worker cannot revoke a database effect already accepted.
- `check_store` verifies all write conditions, refusal preservation, conflict precedence, byte-exact payload, retention expiry equality and concurrent single-winner writes. Adapter-specific queue/lock tests establish protected clock sampling. Passing those tests does not prove deployment durability or historical rollback prevention. ADR-0004

1.6.2 Optional PostgreSQL integration

PostgreSQL record adapter

responsibility

Implement the complete sealed-record store contract in the optional `hellhound_postgres` package.

interface

config(pool name), with_schema(Config, name), with_timeouts(Config, statement milliseconds, lock milliseconds), new(Config), initialize(Adapter), records(Adapter).

interactions

Caller-owned named Pog pool and PostgreSQL schema; core Hellhound has no dependency on this package.

invariants

Only opaque records are stored. Every callback owns its transaction; no borrowed application transaction is committed. Pool lifetime, sealing keys, capacity and backup protection remain application responsibilities.

failure

Configuration and unknown formats are refused explicitly. Unconfirmed writes remain unknown; no fallback, retry or automatic pruning is provided.

- Opaque configuration is built with pure defaults and composable schema/time-limit settings; only new validates the complete value. ADR-0014 Configuration admits a quoted schema name of 1–63 bytes, excluding zero bytes and reserved `pg_` names. Positive lock and statement limits are at most 60 seconds, with the lock limit no greater than the statement limit. new is pure. initialize serializes a versioned schema; unknown versions are refused on initialization and every callback.
- A guarded replacement locks the row, compares its version, then samples PostgreSQL `clock_timestamp` at microsecond precision. Exact numeric nanosecond comparison preserves deadline equality refusal; retention seconds/nanoseconds and sealed bytes round-trip without conversion. Missing rows and changed versions take precedence over time.
- Each transaction sets local statement, lock and idle-transaction limits. These limits do not bound pool checkout, network loss or all cleanup; the caller retains Hellhound's store deadline. A timely decision may commit or be acknowledged later. Known pre-write format refusal is unavailable; uncertain database writes are unknown.

BEHAVIOR

A durable adapter retains opaque records

Given A caller initialized a supported store and committed a record.

When Another adapter reads through the same database.

Then It receives identical bytes, version and retention time.

level: boundary · area: Conditional storage

BEHAVIOR

Unknown storage formats refuse operations

Given The schema format is unsupported.

When A caller initializes or uses an adapter.

Then The operation refuses access without interpreting or mutating record data.

level: boundary · area: Conditional storage

- The public consumer covers the common store conformance suite, fractional expiry, competing writes, protected clock sampling after lock waits, conditional refusals, unknown acknowledgement, client replacement and rotating refresh. It depends on Pog directly, not an application job runner. These receipts establish the tested adapter contract, not production durability, rollback prevention, shutdown completion or security certification.

1.6.3 Session lifetime and installation

Session custody status vacant · live · ended · expired

Custody record aggregate · stateful · Token custody · owned by Hellhound

The mutable authority behind a session reference.

ATTRIBUTES

derived **Status** · Session custody status session-status · session-lifecycle
Derived at read from the live/tombstone record and lifetime policy.

derived **Reference and installation** · digest key and command identifier
Random 256-bit reference and exact owner-bound installation command.

observed **Identity and authentication evidence** · issuer/subject, claims, nonce, auth_time
Verified original provider evidence; refresh cannot create a new authentication event.

observed **Tokens and token revision** · access/refresh/logout hint, expiry, scopes, revision
Accepted provider material; missing expiry remains absent. Publication advances revision.

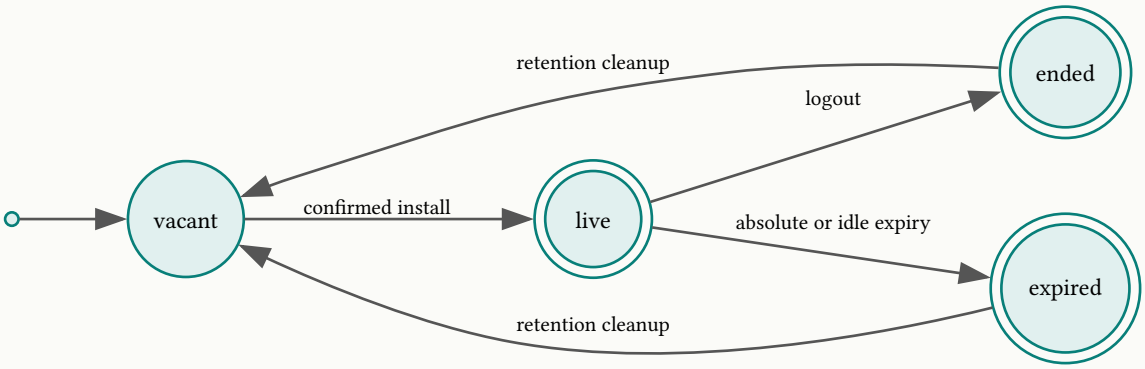
derived **Lifetime and receipts** · created_at, last_used_at, bounded command history
Wall-clock absolute/idle expiry and at most eight recent publication receipts.

RELATIONSHIPS

`n : 1` Each record belongs to one configured provider/custody owner.

`1 : 0..1` At most one outstanding refresh dispatch owns its current generation.

STATE MACHINE Session admission and termination



An ended retained installation cannot return to live through recovery. Expired records are denied before sweeper cleanup.

- Session is an immutable checked view holding client binding, reference, identity and token revision. Custody remains authoritative when a copied view is old.
- Installation inserts once and retains its command identifier. Recovery uses the same opaque command/reference/material; an earlier accepted command returns its current revision. A conflicting ended installation cannot be reinstalled.
- Install commands older than the login-lifetime recovery horizon fail RecoveryExpired. Logout tombstones preserve provider and install evidence for that horizon; late matching recovery reports RecoveryEnded.
- Session expires at either absolute or idle deadline, with equality invalid. Restore, access, refresh and userinfo count as use. Idle touches are coalesced at min(60 s, idle/10); a failed touch does not turn a valid read into a write-confirmed result.
- Record physical retention is at least installation horizon even if live session lifetime is shorter. Sweeper removes records at `expires_at` every 60 s; expiry denies use without waiting for the sweep.
- Session count per identity is unbounded. Applications choose admission/rate/capacity policy; library defaults do not evict an older session to admit a new one. ADR-0004

1.6.4 Refresh generation state and authority

Refresh generation status idle · outstanding · orphaned · quarantined · revoked

Refresh generation entity · stateful · Token custody · owned by Hellhound custody

The dispatch permission and settlement evidence for one custody token revision.

ATTRIBUTES

derived **Status** · Refresh generation status refresh-status · refresh-lifecycle
Confirmed reservation, settlement or publication determines the current permission.

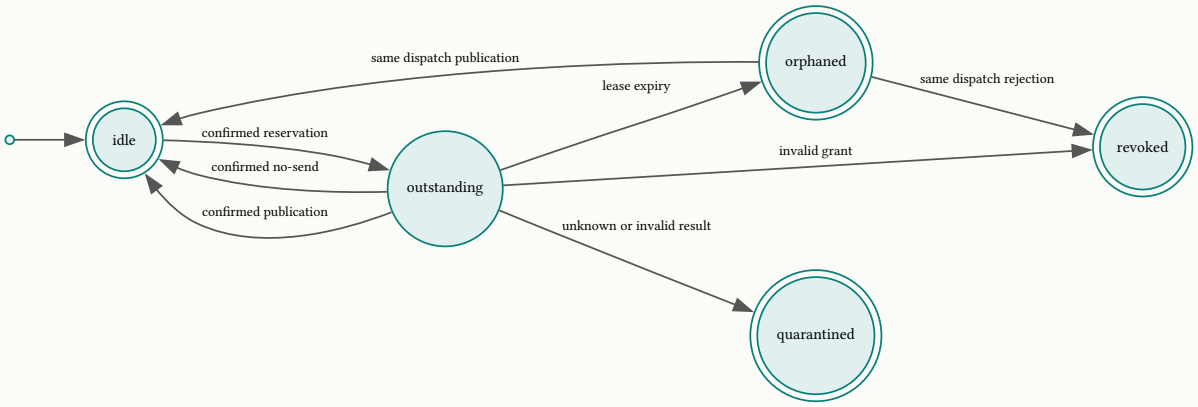
derived **Dispatch binding** · reference, provider, revision, command, dispatch id
A newly confirmed reservation is the only authority for provider invocation.

derived **Lease** · wall-clock expiry
Request timeout plus two store timeouts plus one second, rounded to seconds.

RELATIONSHIPS

`n : 1` A generation belongs to one custody record.

STATE MACHINE Refresh permission and settlement



Lease expiry never releases the predecessor for resend. Its own delayed publication can still install valid new material.

- reserve binds provider, reference, expected token revision and fresh command. A stale revision returns the current snapshot; busy, quarantined, revoked, missing and unknown reservations never dispatch. CAS conflicts retry at most eight times per transition.
- A confirmed NotSent settlement releases an outstanding generation. A known OAuth refusal such as `invalid_request` may permit later retry; `invalid_grant` revokes. Unknown status, possible send or invalid success quarantines.
- An outstanding lease found expired becomes Orphaned. Late no-send cannot release it; only its own valid publication or definite rejection may settle it. No process-local monitor substitutes for this multi-node lease.
- The refresh response may omit ID token and retains original identity. Present ID token must retain issuer, subject, audience, authorized party, original nonce when present and original `auth_time` when present.
- Omitted refresh token or ID-token hint retains prior material; explicit replacement supersedes it. Returned scopes must match the established case-sensitive set; broader scope transitions require a separate profile decision.
- Publication binds dispatch and command, replaces access and refresh material atomically and increments token revision. At most eight recent command/revision pairs permit duplicate publication receipts; old command eviction never authorizes another provider call.
- RefreshUnconfirmed retains exact publication material. `recover_refresh` repeats publication only, and rejects a foreign client. CustodyUnconfirmed and RefreshUnconfirmed have distinct opaque recovery types. ADR-0005

BEHAVIOR

Possible send withdraws refresh retry permission

Given A fresh dispatch authorized one refresh request.

When Its provider outcome becomes unknown.

Then The generation is quarantined and the predecessor is not sent again.

Then The caller action is reauthentication, rather than an ordinary retry.

level: boundary area: Refresh settlement

BEHAVIOR

Publication recovery cannot refresh again

Given New token material exists but publication was not confirmed.

When The caller submits the retained refresh recovery.

Then Custody attempts the same publication and reports its confirmed current session when recorded.

Then No provider refresh is dispatched.

level: boundary area: Refresh publication

1.6.5 Access operation and competing requests

- `access_token` restores the current custody snapshot and returns Access(session, token, `expires_at`, scopes). It refreshes within the default 30-second margin. Missing provider expiry is not fabricated.
- A competing request polls custody with backoff from 20 ms to 250 ms for at most `refresh_wait`, default five seconds. Waiting never creates a second dispatch; another attempt is admitted only after confirmed no-send settlement.
- An old Session uses current custody revision. Forced refresh returns the newer already-published token instead of repeating refresh.
- A margin-triggered refresh with RetryLater may return the still-unexpired current token. Revoked or quarantined outcomes never fall back. Forced refresh never uses that fallback.
- authorize sets Bearer on a caller-owned `gleam_http` request; resource network I/O and resource-request retry remain application-owned. A resource 401 may motivate one forced refresh under application policy.

1.7 Resource validation and provider operations

Checked access claims, service grants and introspection are independent of browser identity.

1.7.1 Local access-token validation

Resource validator

responsibility

Verify one JWT against configured issuer, audience, algorithm, token-type, time and required-scope policy.

interface

resource.new; pure policy setters; verify returns AccessClaims or TokenError; verifier projects to caller token/result/error types.

interactions

Uses Client's key cache and JOSE verifier; unknown signing keys can trigger bounded refetch.

invariants

No per-token provider request on cached keys. Audience mismatch is exposed only after other token validation.

failure

Rejected, WrongAudience, Forbidden and Unavailable preserve distinct caller response meaning.

Check	Default contract	Explicit variation
size and shape	nonempty ≤8 KiB compact signed JWS; JWE refused	none
algorithm and key	configured asymmetric allowlist before key lookup	`with_algorithms`
type	at+jwt or application/at+jwt	`allow_any_token_type`
issuer and time	exact issuer; required strict exp; iat required; nbf/iat tolerance	client clock tolerance
subject and client	sub required; `client_id` optional with azp fallback; jti optional	caller claims decoder
audience	exact singleton configured audience	AudienceIncluded membership
scopes	all configured required scopes present; scope string/list or scp list	`with_required_scopes`

- WrongAudience means an otherwise checked token names another resource; forged, expired or malformed tokens never gain that description. Required scopes are checked after token audience validation.
- AccessClaims exposes issuer, subject, optional client id/JWT id, audiences, scopes and Timestamp expiry/issue time. `decode_claims` preserves the caller's types. It grants no business permission.
- Local JWT validation cannot see immediate token revocation. Short token lifetimes bound that lag; introspection is the explicit availability/network alternative when immediate provider state is needed.
- Relay owns endpoint scope admission and bearer challenges. Its recipe leaves required scopes unset in Hellhound, projects a Principal/Attestation and exhaustively maps ErrorKind through `on_error`. Hellhound has no Relay dependency. ADR-0006

BEHAVIOR

Unavailable keys do not reject or admit a token

Given The validator cannot obtain authoritative issuer keys.

When The caller verifies a bearer token.

Then The result is Unavailable and no access claims are issued.

level: boundary area: Resource validation

1.7.2 Userinfo and service token grants

- userinfo obtains the token `access_token` would use and checks response subject against established session identity. JSON and signed JWT responses are supported; signed userinfo need not have exp, but its signature and relevant issuer/audience rules remain checked.
- UserInfo is a separate opaque subject-bound claim view and `decode_userinfo` returns caller-owned data. It cannot be substituted for VerifiedIdentity.
- `client_credentials` validates requested scopes locally, uses exactly configured confidential authentication and returns ClientToken with opaque AccessToken, optional Timestamp expiry and granted scopes. One call performs one grant; no caching or user session is implied.
- The application decides service-token reuse and cache identity. Missing expiry, concurrency, scope/resource policy and request retries remain explicit service decisions. ADR-0006

Provider operation boundaries

responsibility

Encode grants and userinfo requests and interpret provider results without choosing application resource retries.

interface

userinfo, `client_credentials` and each operation's typed error/describe function.

interactions

Provider metadata, selected authentication, transport and JOSE validation.

invariants

Userinfo subject continuity and configured grant authentication are retained.

failure

Unsupported endpoint/role, invalid scope, provider refusal, malformed response and session error remain distinguishable.

1.7.3 Introspection and logout

- introspect accepts a raw token, refuses >8 KiB locally and treats empty as inactive without a request. It requires configured credentials and an introspection endpoint.
- Introspection returns ActiveToken(TokenInfo) or InactiveToken; provider/network failure stays Error. exp is strict when present and nbf admits configured tolerance; invalid timing is inactive.
- TokenInfo carries optional subject/client id/time/expiry/type, scopes, audiences and opaque raw claim decoding. Introspection has no resource audience argument: the serving application or Relay must compare audience and permission.
- logout removes custody by reference before provider effects, irrespective of a stale Session revision. It retains a tombstone; missing custody may be treated as already signed out.
- Default logout revokes refresh token, or access token if no refresh token, through RFC 7009 and configured authentication. SkipRevocation is explicit. Failed or unsupported revocation never restores local custody.
- ProviderLogout separately reports redirect, no end-session endpoint or refused endpoint. LogoutRedirect keeps the ID-token hint secret under ordinary inspection; `logout_response` emits 303 and no-store. Post-logout redirect policy is validated.
- Third-party initiation belongs to the public reference RP route: issuer/client selection and target-link handling must be validated by the application. Back/front-channel logout is not provided by that route.

BEHAVIOR

Provider logout failure cannot restore custody

Given Local custody termination was confirmed.

When Provider revocation fails.

Then The returned outcome reports revocation failure and the session remains locally ended.

level: boundary area: Session termination

1.8 Assertion signing and credential exchange

Applications compose signed credentials and explicit exchanges independently of OpenID discovery and user custody.

- A Signed assertion is produced locally. An Exchange client sends it to the configured credential endpoint and admits an Exchange token. Applications retain profile and resource-request authority.

Signed assertion

responsibility

Admit private keys and produce a signed JWT with exactly configured claims and protected headers.

interface

assertion.claims and typed setters; with_claim; sign and sign_with_options; to_string; config.signing_key_from_jwk and signing_key_from_pem.

interactions

Shared key admission and public gose JWS APIs; callers supply profiles and time.

invariants

No network, implicit clock, issuer, subject, audience, lifetime or JWT id. Signing never creates verified identity or access claims.

failure

Closed claim, header, size, algorithm and signing errors omit input material.

- Claims and signed assertions are immutable opaque values whose inspection omits their material. Registered setters replace their own prior value, preserve omitted claims, and use strings, string audiences and whole-second Timestamp values. with_claim accepts arbitrary JSON values but rejects registered names and duplicate custom names; a rejected construction remains rejected through subsequent setters. Empty audience lists are refused.
- RS256 is required. Existing admitted RSA, EC and Ed25519 signing families remain available with an explicit compatible algorithm; none and HMAC are unavailable on this surface. JWK use, key_ops and alg constrain admission and signing. Present alg and kid metadata must be strings. PEM conversion failures are normalized with parse failures. RSA requires at least 2048 bits. PEM admission accepts supported private PKCS1/PKCS8 and EC/Ed25519 formats through gose and applies the same refinements. Public-only or unsupported keys fail without their bytes.
- Default sign emits typ JWT and a present key identifier. sign_with_options independently controls optional typ and kid. Protected alg always comes from the explicit algorithm; arbitrary header extensions are excluded. Input payload and resulting compact assertion are bounded to 64 KiB. Applications choose profile-required claims and lifetime, including deliberate backdating.

BEHAVIOR

Configured assertion material is preserved

Given A caller supplies admitted signing material and valid claims.

When The caller signs with a compatible explicit algorithm.

Then An independent verifier observes exactly the configured claims and supported protected headers.

Then Unspecified claims remain absent and nested custom values retain their JSON meaning.

level: boundary area: Assertion construction

BEHAVIOR

Invalid assertion construction fails closed

Given Claim construction, key admission, header selection or algorithm compatibility is invalid.

When The caller attempts to obtain a signed assertion.

Then The operation returns a typed refusal without exposing credential or claim material.

Then Later setters cannot erase an earlier construction refusal.

level: boundary area: Assertion construction

Credential exchange

responsibility

Execute one explicitly configured credential request and admit its decoded token result.

interface

exchange.options, new, supervised, new_request, with_body, with_header, perform; TokenResponse and caller Decoder; token_value and expires_at.

interactions

A dedicated supervised HTTP Gun pool through the existing transport; no discovery, JWKS, login store or custody.

invariants

At most one request per perform; no retries, redirects, token cache or refresh policy. Request and returned credential inspection are redacted.

failure

Configuration, request, transport/send evidence, provider rejection, decoding and empty/unsafe-token failures stay distinct and omit raw responses.

- The exchange client is an immutable handle with stable pool names. new validates options before allocation of a child process. supervised returns its HTTP pool child; the application starts and stops it through the owning supervisor. Calls before start or after stop fail without creating a replacement pool. Independent clients share no pool. Applications retain captured decoder resources for the call; decoder purity and bounded work are application obligations.
- Options expose system/custom PEM trust, explicit destination policy and host allowlist, a positive request timeout up to ten minutes, a response limit up to 16 MiB, header byte limit up to 1 MiB and header count up to 1000. Defaults are public destinations, system trust, ten seconds, 1 MiB response, 16 KiB headers and 100 headers. Existing HTTPS-only, DNS/TLS, request-size and framing rules apply.
- Requests select a bearer assertion or named form assertion, an empty/form/JSON body, and extra headers. Assertion placement is singular and controlled. Form assertions require empty/form bodies and cannot overwrite an existing field. Duplicate headers/form fields and caller overrides of credential/framing headers are refused before transmission. Default Accept is application/json and may be replaced by one caller header. A caller User-Agent replaces the default rather than duplicating it. Content type is derived from the body.
- Every successful response has a 2xx status and a JSON body decoded by the caller's Decoder(TokenResponse). TokenResponse supplies a token string and optional absolute Timestamp expiry; the caller converts provider-specific relative/absolute expiry fields and handles additional metadata through its decoder. Missing expiry remains absent. Empty or C0/DEL/C1 control-bearing tokens fail. A returned token is opaque and redacted; it proves neither identity, scopes nor business permission. Expiry usability and caching remain caller policy.
- Transport validates headers, encoding, body size and completion before status classification. Its failures take precedence over a non-2xx status, including when an error body is excessive or incomplete. An admitted complete non-2xx response, including a redirect, returns only its observed status; provider response bodies and decoding errors are not retained in diagnostics. Transport failures retain NotSent or MaybeSent evidence and closed transport reasons. Decoder exceptions are normalized without retaining arbitrary terms. Credential material never appears in Hellhound events. The generic API establishes no RFC 7523 or RFC 8693 conformance claim. ADR-0021

BEHAVIOR

Explicit exchange succeeds without discovery

Given A supervised exchange client and a valid explicit credential request.

When The endpoint accepts the credential and the caller decoder admits its response.

Then The caller receives a redacted token with exactly the decoded optional expiry.

Then The endpoint receives one request with the selected credential placement and body.

level: boundary area: Credential exchange

BEHAVIOR

Invalid exchange requests never transmit

Given A request has conflicting credentials, invalid framing, an insecure destination or an excessive size.

When The caller performs the exchange.

Then It returns a typed refusal with no transmission authority.

Then No endpoint receives credential material.

level: boundary area: Credential exchange

BEHAVIOR

Exchange failures retain evidence without resend

Given An exchange is refused, interrupted or returns an inadmissible response.

When The call settles.

Then The caller receives a distinct provider, transport or response failure without secret material.

Then No redirect or automatic resend occurs, and transport uncertainty remains explicit.

level: boundary area: Credential exchange

BEHAVIOR

Exchange pool lifetime belongs to its supervisor

Given An exchange handle exists independently of its pool process.

When Its owning supervisor starts, restarts or stops the pool.

Then The same handle addresses the currently supervised pool.

Then A missing pool never causes an exchange to start an unowned replacement.

level: boundary area: Exchange lifecycle

- Correctness uses the pinned RFC 7520 RS256 vector and independently verifies public API output with test-only erlang-jose and Node crypto. Security cases cover key policy, exact claims, collisions, redaction, transport bounds and uncertain outcomes. Two compiled recipes demonstrate GitHub-style bearer/JSON exchange and OAuth-style form assertion exchange. The OAuth recipe requires a recognized Bearer token_type, accepts it case-insensitively, and distinguishes omitted expiry from malformed or null expiry. External corpus inputs are fetched at an immutable revision with a recorded digest; upstream tests inform cases without becoming runtime dependencies. The test progression is failing behavior, minimal implementation, then refactoring with tests passing.

1.9 Transport and observations

Provider traffic and diagnostics preserve trust, limits and send evidence.

Hellhound to HTTP Gun contract

Carry provider operations over bounded verified HTTPS.

responsibility

Hellhound applies OIDC URL/header/body policy; HTTP Gun owns DNS, destination checks, connection and framing.

interface

Supervised shared HTTP Gun client and per-request deadline/response limits.

interactions

One pool per Hellhound client; configured trust anchors, hosts, destination policy and Sinal correlation.

invariants

No redirects, automatic grant retries or decompression. Requests use selected provider endpoints only.

failure

NotSubmitted becomes NotSent; possible submission becomes MaybeSent. Typed transport failures omit provider body.

- HTTP Gun resolves once per connection, checks every destination address before connection, connects to checked addresses and verifies TLS against the original hostname or IP SAN. Production defaults allow public addresses and system trust; loopback/internal providers need explicit policy.
- Host allowlists constrain hostnames, not ports. Certificate revocation checks are not claimed; deployment egress policy remains a separate owner.
- Hellhound rejects insecure schemes, userinfo/control characters, unsafe headers and requests above 64 KiB. It refuses oversized declared length early and rejects content encoding; provider response default is 1 MiB with 16 KiB head and 100 headers.
- Transport limits bound collected input, not total VM/kernel/TLS heap. Decoded key sets and forwarding add allocations beyond JSON byte size. Concurrent active clients multiply those costs.
- Four malformed responses can map to ReceiveFailed with possible-send evidence rather than finer parse classes. Bare LF in discarded status reason text may be accepted on a close-delimited connection. These existing limitations are retained by ADR-0002.
- Unknown foreign shapes and exceptions become sanitized typed failures. Hellhound implements no cryptographic primitive and contains supported foreign boundary failures; no arbitrary formatter turns raw provider terms into diagnostics.

1.9.1 Typed observations and correlation

Telemetry

responsibility

Report completed provider/login/refresh/logout outcomes using closed redacted values.

interface

hellhound/telemetry descriptors and pure `with_correlation` client view.

interactions

Sinal emits over native telemetry; correlation is also passed to HTTP Gun.

invariants

No token, code, state, nonce, verifier, identity, query, body or header enters Hellhound events.

failure

Observer failure does not authorize a state change; callbacks run synchronously and can add latency.

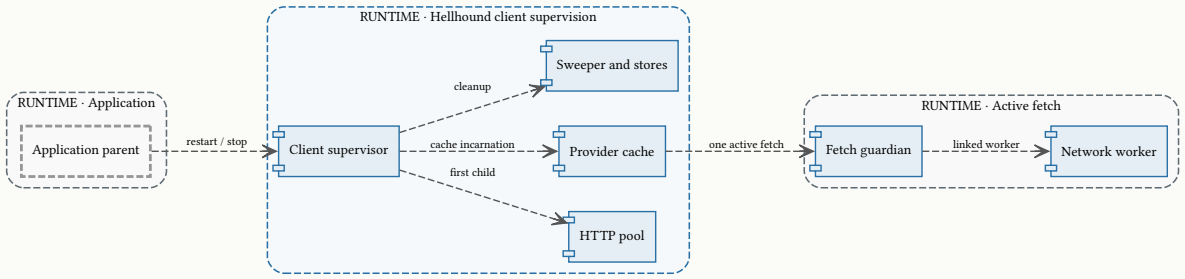
Descriptor	Emission point	Permitted data
`http_request`	provider request returns or fails	duration, method, host, path, status or evidence/reason, correlation
login	`complete_login` or `recover_custody` returns	duration, closed outcome, correlation
refresh	refresh completes, joins or fails	duration, closed outcome, correlation
logout	custody has ended	duration, closed revocation outcome, correlation

- `with_correlation` returns a value view and shares all client processes. Caller-supplied correlation is not authentication, a session key or idempotency authority.
- Opaque redaction prevents ordinary inspect output; it does not erase BEAM memory or defend against `sys:get_state`, tracing, crash dumps, malicious trusted callbacks or host administrators. Applications protect these operational surfaces.

1.10 Lifecycle and limits

Stable handles survive process replacement; admitted effects retain their own cleanup boundary.

VIEWPOINT RUNTIME · ALTITUDE L3 · COMPONENTS Per-client process ownership



LEGEND



The guardian monitors the actual cache and worker. Durable stores are outside this tree and are never joined by startup.

Client lifecycle

responsibility

Create and replace only the client's own processes without stale key or name authority.

interface

new, start, supervised and stop.

interactions

Application parent, pool, provider cache, sweeper and default stores.

invariants

Names are allocated once at trusted setup; request strings never become atoms. Live supervisor means AlreadyStarted.

failure

Startup joins held old children under one deadline; expiry returns StartupTimedOut without takeover. stop is a bounded request, not a completion receipt.

1.10.1 Startup and cache replacement

- Manual start builds the tree and awaits the cache's first compatible discovery plus keys. First-attempt outcome remains metadata or failure, never a historical key set, so a background recovery cannot hide that initial failure.
- Supervised starts discover in the background. Until ready every relevant operation fails closed. Restart-stable client names do not imply ready metadata or persistent default sessions.
- Whole-tree replacement monitors the actual old pool, cache, sweeper and optional in-memory stores before using their names. Startup cleanup and manual discovery share one monotonic deadline. At most five temporary monitors are removed on exit or expiry.
- A held old child can delay its parent's synchronous startup management up to `startup_timeout`. Existing unrelated client request processes remain independent; no global coordinator serializes them.
- Failed manual start requests shutdown only of the supervisor it created and waits using the remaining budget. Timeout can return while that tree drains, so immediate retry may report AlreadyStarted. ADR-0007

1.10.2 Fetch ownership and cancellation

- Each active fetch has one guardian beside its network worker. The guardian monitors the actual cache and worker and links the worker; cache monitors guardian. There is no idle guardian.
- Completion goes to the cache process subject with guardian identity. Obsolete completion/exit notifications cannot settle newer work or reach a replacement's registered name.
- Cache death terminates the worker even when HTTP or an application telemetry handler blocks. Worker loss releases active state and waiting callers, preserves last accepted snapshot and follows existing retry/reload cadence.
- One active fetch adds one process, three monitors, one link and one forwarding hop. The decoded metadata/key result is copied once more; cost depends on key-set size and is not bounded by JSON bytes alone.
- Cancellation is process-ownership loss and deadlines rather than a public cancellation-token API. Termination cannot undo a provider request already received. ADR-0007

BEHAVIOR

Old fetch completion cannot alter a replacement

Given A fetch's cache incarnation has ended.

When That fetch completes.

Then Its result cannot become the replacement cache's provider snapshot.

level: boundary area: Cache lifetime

1.10.3 Public defaults and setters

Operation	Default	Setter
`start`: discovery, first keys and previous-process cleanup	15 s	`config.with_startup_timeout`
`supervised`: background discovery	retries from 1 s to 60 s	none
provider request	10 s; HTTP Gun connect 5 s, pool checkout 5 s, idle read 30 s, idle connection 60 s	`config.with_request_timeout`
provider response body	1 MiB; head 16 KiB, 100 headers; no redirects	`config.with_max_response_bytes`
request body sent	64 KiB	none
bearer token introspected or validated	8 KiB	none
store call	5 s	`config.with_store_timeout`
provider-cache call	request timeout + 1 s	none
`complete_login` effect admission	30 s	`config.with_login_timeout`
pending login lifetime	10 min	`config.with_login_lifetime`
built-in login store records	100 000	`config.with_max_pending_logins`
callback input and login option values	16 KiB total and 4 KiB per callback value; 2 KiB per option value	none
session	12 h absolute, 1 h idle; sessions per identity unbounded	`config.with_session_lifetime`
refresh margin before expiry	30 s	`config.with_refresh_margin`
wait for another request's refresh	5 s	`config.with_refresh_wait`
refresh lease	request timeout + 2 × store timeout + 1 s	none
resend after a possible send	never, for authorization codes and refresh tokens	none
logout revocation	refresh token revoked under RFC 7009	`LogoutOptions(revocation:)`
install-recovery horizon and logout tombstones	the login lifetime	none
expired-record sweep	every 60 s	none
key cache	`Cache-Control` clamped to 60 s–24 h, else 1 h	none

Operation	Default	Setter
unknown-`kid` key refetch	at once per new kid (64 remembered), else once a second	none
supervisor restarts	10 in 60 s	none
clock tolerance for `iat`, `nbf` and `auth_time`; never `exp`	5 s	`config.with_clock_tolerance`
ID-token and access-token algorithms	RS256, PS256, ES256, EdDSA; `none` and HMAC unrepresentable	`config.with_signing_algorithms`, `resource.with_algorithms`
PKCE	advertised S256 required	`config.with_pkce_advertisement`
RFC 9207 `iss`	required when advertised	`config.with_issuer_parameter`
destinations and TLS trust	public addresses, system trust	`config.with_destinations`, `with_allowed_hosts`, `with_trust`
ID-token audience	exactly the client	none
access-token audience and type	exactly the configured audience; `typ at+jwt`	`resource.with_audience_policy`, `allow_any_token_type`
binding cookie	`__Host-hellhound_binding`, `Secure`, `HttpOnly`, `Path=/`, `SameSite=Lax` (`None` for form post)	none
session and login storage	in memory	`config.with_custody_store`, `with_transaction_store`
`client_credentials`	not cached; one token request per call	none

- This table owns the public defaults. A caller overrides a default only through its setter; no default is read from the environment. ADR-0018

1.10.4 Bounds and cleanup guarantees

Boundary	Bound	Expiry or cleanup meaning
provider request	10 s default; HTTP Gun connect/checkout 5 s, idle read 30 s, idle connection 60 s	typed failure retains send evidence; inner bounds can expire earlier
provider-cache call	request timeout +1 s	typed not-ready/failure; no fabricated key rejection
complete login	30 s admission deadline default	remaining time passed to each effect; cleanup/ telemetry caveat below
store call	5 s worker deadline default	kill worker, wait DOWN up to 1 s, drain late reply; accepted external effect can remain unknown
refresh wait	5 s default	RefreshWaitTimedOut; no dispatch permission
refresh lease	request +2×store timeout +1 s	Orphaned quarantine; not resend permission
stop	5 s supervisor observation	Nil on exit or expiry; worker notification can follow supervisor exit
storage sweeping	60 s cadence	logical expiry already denies use

- The complete-login deadline bounds effect admission and propagated time budgets. A timed-out store's monitor drain can add up to one second, and synchronous telemetry observer work has no Hellhound timeout; the pending verification records this precise return-latency tension.
- stop neither confirms shutdown on wait expiry nor synchronously joins fetch workers or external adapters. Stop a supervised client through its owning parent or it may restart. Applications retain captured resources until their required completion boundary is known. ADR-0011
- Built-in pending-login capacity counts pending and retained terminal records. FIFO/capacity bookkeeping avoids unnecessary scans while there is room; a full login store returns TooManyPendingLogins. Sessions deliberately have no identity-count cap.

1.11 Failures and caller decisions

Typed failures preserve phase, authority and effect evidence.

Family	Distinct evidence	Caller handling
StartError / ConfigError	invalid config, discovery failure, incompatibility, startup timeout, process failure, already started	describe safely; fix configuration or apply deployment startup policy
LoginError	malformed/refused callback, expired/ replayed/changed transaction, store failure, denial, exchange no-send/ unknown, rejected identity, unconfirmed installation	`login_error_action`
SessionError	missing/lost/foreign/unreadable/ unavailable; absent access/refresh; revoked/refused/no-send/quarantined; wait timeout; unconfirmed publication	`session_error_action`
TokenError	shape/type/algorithm/signature/keys/ issuer/time/claims/audience/scopes	resource.`error_kind` and caller projection
ProviderFailure	not ready; TransportFailure(evidence, reason); status and closed OAuth code; malformed response; issuer mismatch; unmapped failure	preserve operation-specific effect meaning
StoreError	unavailable versus outcome unknown versus full	adapter must report knowledge honestly
Userinfo / client credentials / introspection / logout	role/endpoint/option/session/provider failures	operation-specific typed result and describe function

Closed Action	Meaning
Reauthenticate	Start a new login; never replay a possibly sent code or quarantined refresh token.
RetryLater	No final decision; repeat only the operation whose contract permits it.
Recover	Submit carried custody or refresh publication recovery; never call provider again.
FixConfiguration	Operator/configuration issue such as foreign owner or unsupported authentication.
RejectRequest	Incoming request is invalid and should not be retried.

- Error unions and protocol registries may gain variants; Action is the stable closed handling boundary. Use describe functions rather than inspecting raw error material.
- A provider status is not universally definite non-action. Refresh maps only recognized safe refusal evidence to release; possible processing and invalid success quarantine.
- Claims or endpoint error descriptions are never embedded in ordinary errors. Known claim names and missing required scope names are safe typed diagnostics, not raw provider payloads.

1.12 Application responsibilities

Hellhound owns checked evidence and token custody; the application owns every decision that evidence authorizes.

1.12.1 Routes, origin and browser session

- The application mounts login, callback and logout routes at the configured **Public origin**. Every exact callback URI is registered with the issuer, and per-login redirects come only from the admitted allowlist. A reverse proxy preserves the intended public scheme, host and path.
- The application passes the incoming request to `begin_login` and the configured query or form-post request to `complete_login`. It uses `login_response` and `logout_response` for Hellhound's response headers. A form-post body is admitted only on the callback route, with a body limit enforced before buffering.
- The **Browser binding** cookie stays intact. It binds a callback and is not the application's authenticated session, so the application chooses its own protected session cookie, lifetime and origin policy and prevents session fixation when login completes.
- The application protects state-changing routes, including logout, with its own CSRF and origin policy, and rate-limits login admission, callbacks and session use. Hellhound's pending-login capacity bounds neither all application traffic nor session count.

- A third-party initiation route validates issuer and client selection and the target link. A URL from that request cannot establish a redirect or permission policy.

1.12.2 Identity and business permission

- The authentication key is the pair of issuer and subject. An email claim is neither a stable identity key nor proof of permission.
- The application defines its users, tenants, roles, resource ownership and approver policy. It projects Verified identity or caller-decoded claims into those types after login and checks permission on every protected operation. ADR-0006
- A resource server chooses its accepted audience, algorithms, token type and scope policy. Local JWT validation cannot observe immediate provider revocation, so a deployment that needs that authority uses introspection and checks audience and permission on the returned facts.
- Under Relay, Relay owns audience, required-scope and challenge policy for its admission path. Provider unavailability stays distinct from rejected bearer evidence.

1.12.3 Custody and persistence

- The application retains the Session reference in its protected session and restores it for later requests. The reference is a bearer secret. Provider access, refresh and ID-token values stay out of cookies, URLs, logs and ordinary error pages.
- The application chooses in-memory or durable stores before startup. In-memory custody is lost on restart. A durable adapter implements all three write conditions, reports unavailable and unknown outcomes honestly, and passes `testing.check_store` plus deployment-specific race, crash and durability checks.
- The application protects database writes and backups against record rollback, because sealing authenticates a record without establishing monotonic history after whole-row restoration.
- Sealing keys come from trusted secret storage. The current and previous key ring is distributed consistently across nodes and rotated before nonce use becomes unsafe. A destroyed required old key makes its records unreadable.
- Host clocks are synchronized. Durable login and session lifetimes use wall-clock time, operational budgets use monotonic time, and a guarded write samples the adapter's documented clock after exclusive write authority. ADR-0013

1.12.4 Refresh, retry and logout

- The application calls `access_token` for the current usable user token. Concurrent refresh belongs to Hellhound, and callers neither reserve, resend nor repair a rotating token.
- The application handles failures through the closed Action projections. Reauthentication follows terminal login failure or refresh quarantine. `Recover` submits the carried custody or publication recovery, which retries publication and never the provider exchange.
- After a resource response, the application decides whether its request is safe to repeat. A forced `refresh` fits only where the application holds that retry authority and does not make a resource effect idempotent. A possibly sent authorization code or refresh token is never retried.
- `client_credentials` performs one grant per call, returns an optional expiry and creates no user session. The application sets its own cache policy for service tokens, including the policy for an absent expiry.
- On logout the application removes its own session, and inspects local custody failure separately from the returned provider revocation and redirect outcomes.

1.12.5 Resource lifetime and observability

- The application creates client handles once at trusted setup and supervises them under its own parent. A supervised client stops through that parent. A cache restart needs fresh discovery, so a stable handle does not imply provider readiness.
- Adapter connections and other captured resources stay alive until every admitted operation reaches the completion boundary the application requires. `stop` is not a completion receipt for fetch workers or external effects.
- The `complete_login` admission deadline is not a strict return-latency guarantee, because timed-out store cleanup and synchronous telemetry add latency. The application bounds its request handling accordingly.
- The application logs through the `describe_*` functions and typed Sinal events. Correlation carries no authentication authority. Telemetry handlers stay bounded because they run synchronously. Tracing, crash dumps, VM inspection and privileged host access need separate protection.
- Local test-provider trust is used only in tests. Production endpoints require the intended issuer, destination policy and TLS trust. Host allowlists restrict names, so outbound ports need deployment network policy.

1.13 Testing and retained capability scope

Executable consumers and independent oracles establish specific contracts with explicit evidence limits.

1.13.1 Test and extension ports

Public testing support

responsibility

Provide explicit local issuer behavior and store-conformance checks to consumers.

interface

hellhound/testing scripted provider, PKI, authorize, token issuance/mutation, request counts and `check_store`.

interactions

Linked HTTPS listener/state actor using OTP ssl/`public_key` and gose; application routes need no production test hook.

invariants

Only explicit testing.config/trusting grants loopback and test-root trust. Production defaults never weaken.

failure

Typed startup/test failures; caller/link lifetime cleans provider processes; fixtures still do not certify security.

- Typed EmailClaims pairs independently optional email and verification assertions. `with_email_claims` configures one subject; `set_email_claims` changes that subject's future ID tokens and userinfo evidence. None omits the field, while Some(False) preserves explicit false. These assertions do not validate email or confer application permission.
- A subject without an override retains subject-derived email and verified true in ID tokens; userinfo retains subject-derived email without a verification field. Reserved protocol claims, production trust and opaque identity construction remain unchanged. Previously signed tokens and immutable verified identities do not change when fixture settings change. ADR-0012

BEHAVIOR

Scripted email assertions retain absence and subject scope

Given The local test issuer has explicit email assertions for one subject.

When It issues identity or userinfo evidence for that subject.

Then Each assertion is present or absent exactly as configured, including explicit false.

Then Subjects without overrides retain the issuer's existing default assertions.

level: boundary area: Public testing support

BEHAVIOR

Changing fixture assertions preserves prior identity evidence

Given A client already holds checked identity from the local test issuer.

When The fixture changes that subject's email assertions.

Then Later evidence reflects the new assertions.

Then The identity held earlier and its issuer/subject key remain unchanged.

level: boundary area: Public testing support

- The provider supplies discovery/JWKS, S256 code login, rotating refresh, service grants, userinfo, introspection, revocation and end-session. It scripts sign-in/refusal, clock skew, audience/key changes and forged tokens.
- Scripted scopes are per subject: configured list replaces requested scopes exactly, including empty or unrequested scopes; no entry preserves requested scopes. One grant feeds token response, JWT and introspection.
- Testing trust exposes PEM for Hellhound and DER for HTTP Gun. Test listener/state are linked to their caller; tests stop supervised fixtures through their parents.
- Compiled consumer exercises common login/access/logout, advanced authentication/transport policy, custom CAS storage and caller-owned claims/errors. It also holds one runnable usage example per topic (supervision, durable stores, callback addresses, resource servers, the local test provider and application decisions), and the README shows their marked regions verbatim. Relay consumer verifies admission/challenge behavior.
- Negative compiler fixtures reject forged opaque authority and interchange of identity, userinfo, introspection, session, access token and recovery phases. Runtime tests establish race, uncertainty, redaction, bounds and process ownership beyond those type checks.

1.13.2 Evidence and release conditions

BEHAVIOR

Disposable provider cleanup requires ownership

Given A fixture command encounters a listener or a retained ownership record.

When It starts or stops a disposable Node provider or Hydra consent fixture.

Then Startup refuses an occupied address or an existing owner record.

Then Shutdown requires the fixture's private control endpoint and matching ownership token; a public port or PID file cannot authorize a signal.

Then Failed or interrupted startup cleans its own process and record; malformed or unreachable ownership state is retained for inspection.

level: boundary area: Provider fixture lifecycle

Evidence family	Source and operation	Acceptance limit
Supply chain	major-bounded runtime ranges and exact lock; pinned oidcc 3.9.0/jose dev oracles; license records	review changed resolved versions and advisories; runtime pins do not inherit certification
Owned facade	URI/callback/scope tables, RFC7636 vector, races, CAS, unreadable records, secret sentinels	type opacity alone proves neither secrecy nor store atomicity
JOSE / differential	gose/kryptos, independent panva/jose tokens, RFC7515/7517/7520 and Wycheproof vectors, raw oidcc corpus; RFC9449 vectors when DPoP is built	record strict audience/required-ID-token/refresh differences; wrapper tests do not revalidate all cryptography
Provider interoperability	Keycloak, node-oidc-provider, Dex, Hydra; recorded receipts	local fixtures do not establish arbitrary-provider compatibility
OIDF RP plans	Basic, Config, Form Post, refresh, RP logout and third-party initiation harness	skipped/review/interrupted cases are not passes or certification
Operational	restart, fetch owner loss, barriers, bounds, lease loss, mailbox/atom/memory tests	local benchmark samples are not deployment capacity
Replacement	same contracts, corpus, differential, selected conformance profiles	never shadow single-use code or rotating-refresh effects

- Release acceptance requires no vulnerable/unreviewed dependency, no owned-facade failure, no unexplained differential, no selected conformance regression, no failing attack requirement and no supported-provider interoperability failure. Independent review and release SBOM remain obligations.
- Raw license notices, vendor record declarations, lockfiles, provider image/schema fixtures, receipt JSON/logs and executable probes remain intact. ADRs carry provenance and interpretation rather than rerun diaries. The pins, comparison classification, deliberate differences and reproduction commands are in ORACLE.md. ADR-0008

1.13.3 Validation gates and continuous integration

- Required checks cover formatting, compilation and unit/integration behavior. Bespoke README equality checkers, source-policy audits, gate self-tests and repository-local design checkers are removed; shared pinned Snips synchronizes README excerpts from compiled and tested examples; the imported design check remains. Documentation and conventions are reviewed manually. ADR-0019

Obligation	Command	Cadence	Enforcement and retained evidence
Shared formatting	`nix flake check`	direct local command, mandatory CI	mechanism; formatter diagnostics
Design integrity	`nix run .#design-gate-check -- docs/design .`	design changes, mandatory CI	imported checker

Obligation	Command	Cadence	Enforcement and retained evidence
Gleam build and authored Erlang warnings	root build sequence below; unflagged consumer build; `scripts/erlang-check`	local gate, mandatory CI	mechanism for compiler diagnostics
Native attacks, races, redaction and lifecycle	`HELLHOUND_SUITE=fast gleam test`	local gate, mandatory CI	bounded runtime evidence; nonempty known suite selection
Opaque constructors and public composition	`scripts/negative`, consumer build and tests, `scripts/relay-recipe`	local gate, mandatory CI	compiler rejection plus positive control; compiled Relay consumer
Usage examples	consumer `usage_test`	local gate, mandatory CI	examples compile and run against the local provider
PostgreSQL adapter	`scripts/postgres-check`, `scripts/postgres-negative`	local gate, mandatory CI	public store adapter and clients against a temporary cluster; opaque-configuration compiler fixture
Independent providers	`scripts/provider-check node`, `keycloak`, `interop`, `browser`	relevant changes, daily, full local gate	bounded interoperability and browser evidence; logs and browser JSON
Selected OpenID plans	`scripts/conformance-check` with the regression policy and selected plans	manual OpenID workflow, release selection	bounded regression evidence; raw module logs and acceptance summary
Cache and resource costs	`scripts/benchmark`	optional manual benchmark workflow	observations only; environment and measurements, no latency threshold

- The test-only JOSE 1.11.13 oracle includes upstream security and OTP 29 fixes. Root and consumer builds run `gleam build --warnings-as-errors`, and authored Erlang compiles with `erlc -Werror`. No deprecated-catch compiler exception is required. ADR-0022 supersedes the exception in ADR-0016.
- Mandatory CI runs the canonical local gate on pull requests and pushes to `main`, and its final status rejects failed, cancelled or skipped mandatory jobs. CI builds Sinal, HTTP Gun, Relay and JSON Blueprint from Hex at the versions and checksums locked in each `manifest.toml`, with read-only repository permissions and no persisted credentials. ADR-0015 ADR-0020
- CI caches Hex archives and path-dependency fingerprints but no compiled output. Dependency preparation retries explicit Hex rate limits at most three times. Compiler and test failures are never retried.
- OpenID verdicts default to strict, so every selected module finishes PASSED. Regression mode permits only the exact SKIPPED unsigned-token and REVIEW logout outcomes in `test/conformance/known-outcomes.json`, which remain non-pass evidence. Third-party interruption, unknown outcomes, driver errors, missing modules, malformed evidence and empty selections fail both policies. Neither policy confers certification.
- The conformance driver archives a terminal or unready setup status and its module log before starting the reference relying party. The driver is used by the optional provider-conformance workflow.
- Each workflow retains bounded artifacts with package and sibling revisions and logs, and provider jobs retain image and browser identities. Artifacts exclude generated PKI and private keys, and a run never overwrites checked-in raw history.
- Nix pins all tooling through `flake.lock`. The formatter excludes frozen evidence receipts, deliberate negative compiler fixtures and generated output.
- Repository documentation consists of the design layer, the README files, `AGENTS.md`, the compiled usage examples, the oracle references and the benchmark measurements. ADR-0010

1.13.4 Dependency provenance and release qualification

Scope	Authority and retained evidence
Runtime cryptography	public gose and kryptos APIs; exact resolution and checksums in `manifest.toml`

Scope	Authority and retained evidence
HTTP and observations	HTTP Gun and Sinal Hex version requirements, locked by their manifests, with licenses and package-local design layers
Erlang and Gleam runtime dependencies	package declarations, resolution manifest and licenses supplied with exact sources
Differential Oracle	exact test-only oidcc pin; record declarations included from the oidcc development dependency
Scripted provider signing	exact test-only erlang-jose pin; provider fixture lockfiles and notices
Independent providers	Keycloak image digest, node-provider package locks, OIDF compose image pins and retained harnesses
Raw qualification evidence	each Evidence receipt under `docs/evidence/`; interpretation in ADRs

- `gleam.toml` owns admitted dependency ranges, and `manifest.toml` records one concrete resolution that does not qualify every admitted version. Runtime and test-only dependencies stay separate: oidcc, erlang-jose and telemetry_registry are development oracles that an application does not install by using Hellhound.
- A release records the actual package, sibling, dependency, fixture and toolchain revisions and replaces unpublished sibling Git dependencies with approved release declarations. It reviews resolved upstream changelogs, licenses and advisories, produces the release SBOM, and checks advisory-query coverage against known controls. An empty query result never establishes absence of vulnerabilities.
- A changed dependency or fixture pin runs the relevant provider, conformance and compatibility suites. Failed, skipped and human-review results stay distinct from passed assertions, and an older receipt never qualifies a new resolution or pin. ADR-0008

1.13.5 Benchmark method

- The retained benchmark entry points are `hellhound_fetch_ownership_test:benchmark/0` and `snapshot_benchmark/0`, run from `gleam shell` with four schedulers. Each starts its own local HTTPS test provider and stops each client fixture. `benchmark/0` warms 100 validations per client, measures 3 000 per client for one and four clients, reports throughput and latency percentiles, and measures 100 sequential key refreshes. `snapshot_benchmark/0` warms 1 000 reads per cache and measures 20 000 per cache for one and four caches.
- Benchmark output is an observation of one checkout on one machine, never a deployment capacity guarantee or a correctness-gate threshold. A reported result retains its raw output, source and dependency revisions and machine configuration. The retained measurements are in BENCHMARKS.md, which links to the oracle references. ADR-0017

1.13.6 Threat and authority coverage

Threat	Invariant owner	Required evidence
login CSRF / fixation	transaction state and binding; app session/origin protection	binding mismatch leaves pending, protected-route tests
code replay / injection / downgrade	CAS consumption, S256, nonce, selected auth	concurrent single-winner and PKCE/nonce attack cases
issuer mix-up / token substitution	exact issuer, audience, azp and nonce	wrong issuer/audience/party and duplicate callback cases
signature / algorithm confusion	allowlist before keys; gose and key policy	none/HMAC, bad signature, wrong/weak keys, rotation and JWE rejection
malicious discovery / SSRF / TLS	configured destination policy and HTTP Gun	private/mixed/rebinding, redirect, wrong host/expired CA tests
rotation replay / uncertain effects	custody dispatch, quarantine and publication	lost provider response, invalid continuity, orphan lease, recovery without resend
storage forgery / rollback	AEAD and CAS; app write/backup protection	swap/tamper/key rotation; explicit whole-row rollback limit
secret/atom/resource exhaustion	redacted boundaries, stable names, limits; app admission	secret sentinels, hostile callbacks, bounds and process cleanup

1.13.7 Full capability disposition

Capability	Disposition	Growth or evidence contract
discovery/JWKS; code/S256/state/nonce; five authentication forms	supported	provider/cache, login and verification units
identity/claims; sealed custody; refresh/recovery; service grants	supported	typed authority, CAS and uncertainty units
JSON/JWT userinfo; introspection; revocation/RP logout; local JWT resource role	supported	operation-specific checks and caller authorization
query/form callback; third-party initiation reference path	supported path with third-party suite gap	own HTTP browser/session obligations and external verification
PAR, JAR, DPoP, JWT-bearer grant, dynamic registration	retained later intent	separate typed protocols, authority and external suites
JARM; FAPI2 security and message signing	optional later intent	profile-specific owner decision and conformance
device grant, token exchange, back/front-channel logout	explicit unsupported gaps	growth decision and protocol evidence
social-provider quirks; broader refresh scope/resource profiles	optional / unresolved	concrete provider/consumer need; no implicit generic extension
replaceable protocol backend and application mocks	retained architectural replaceability; no public backend selector	protocol operation contracts plus same independent corpus
stateless cookie-only take-once; signing-key trust anchor	unsupported	AEAD lacks replay authority; no offline-key requirement
authorization server; implicit/hybrid/password; JS runtime	excluded	no inference from oracle capability

- Backend replaceability is an architecture obligation, not an exposed alternate selector. Sole runtime backend is native Gleam over gose/kryptos; oidcc is test-only. Any replacement preserves opaque constructor ownership and operation-specific contracts. A default replacement after published use retains the predecessor for at least one release; shadow comparison is limited to side-effect-free verification/parsing. ADR-0001
- The retained scope is not narrowed to today's signatures. Unbuilt families remain in Pending updates; runtime/example changes require separate evidence and authorization. ADR-0009

1.14 End-to-end walkthrough

A relying party signs Ada in, keeps a session reference, and handles one resource request.

- The application creates a relying-party Config for https://login.example.com and a registered callback, then calls new and supervised. Provider unavailability produces typed not-ready responses until compatible discovery succeeds.
- Ada follows the application's login route. `begin_login` inserts fresh transaction material; `login_response` sets Hellhound's binding cookie and sends the authorization redirect.
- The provider returns a bound code to the callback. `complete_login` parses, compares state/browser/issuer, consumes by CAS, exchanges once, checks the required ID token and installs sealed custody. The application retains the reference in its protected session.
- A later request restores the reference and calls `access_token`. Near expiry one request reserves refresh; competitors wait. If provider outcome is unknown they reauthenticate; if publication acknowledgement is uncertain they submit its exact recovery without another refresh.
- The application authorizes its own API request using the returned AccessToken and decides resource retry after a resource response. Checked identity and scopes are projected into application permission rules.
- Logout first ends custody and retains its tombstone, then reports provider revocation and RP redirect separately. Application removes its own session and stops its owning client supervisor according to its resource lifetime policy.

Glossary

Every term the layer declares, with the context that owns it.

Access claims

owned by root

An opaque checked JWT access-token view for one validator's issuer and resource policy. Application authorization remains a separate decision.

Action

owned by root

The closed caller response classification Reauthenticate, RetryLater, Recover, FixConfiguration or RejectRequest. It classifies recovery meaning rather than generic retryability.

Browser binding

owned by root

The random cookie value tying pending login transactions to one browser. It is distinct from the application's authenticated session.

Cache incarnation

owned by root

One live provider-cache process with its own accepted snapshot and fetch ownership. A replacement process has a different incarnation even when its registered name is unchanged.

Client

owned by root

An immutable handle binding one validated role configuration to restart-stable process names. A correlated client view shares those processes.

Custody

owned by root

Hellhound's authority over session identity, authentication evidence and token generations in sealed records. It owns installation, refresh transitions and termination.

Custody recovery

owned by root

An owner-bound opaque installation command retained after uncertain custody acknowledgement. It can repeat installation of the same evidence without repeating code exchange.

Evidence receipt

owned by root

A retained raw output of one check run with its exact revisions. It bounds the claim it supports and neither certifies a release nor qualifies a later dependency resolution.

Exchange client

owned by root

An immutable handle binding explicit transport policy to an application-supervised HTTP pool, independently of OpenID discovery and user custody.

Exchange token

owned by root

An opaque provider credential admitted by a caller-owned response decoder, with optional observed expiry. It carries no checked identity or business permission authority.

Login transaction

owned by root

A single-use pending authorization-code login bound to one client, exact redirect, state, nonce, PKCE verifier, browser binding and expiry.

Oracle

owned by root

A pinned independent implementation, token corpus or conformance suite whose behavior bounds a named Hellhound contract. It is never a runtime dependency and confers no certification.

Provider

owned by root

The configured OpenID issuer and the endpoints and signing keys authenticated through its discovery and transport policy.

Provider snapshot

owned by root

The metadata and signing-key set accepted by one cache incarnation. It is observed provider evidence, not a substitute for configured policy.

Public origin

owned by root

The scheme, host and port at which browsers reach the application's login, callback and logout routes. Registered callback addresses are built from it.

Quarantine

owned by root

Custody's refusal to dispatch a token generation whose provider outcome or accepted response is unsafe to reuse. It retains uncertainty rather than proving revocation.

Refresh dispatch

owned by root

The fresh custody reservation authorizing one provider refresh call for one token revision. Possession of an old session or a waiting result does not create it.

Refresh lease

owned by root

The wall-clock interval during which an outstanding refresh reservation may settle normally. Expiry withdraws permission to resend the predecessor.

Refresh recovery

owned by root

An owner-bound opaque publication command holding already received refreshed material. It cannot authorize another provider request.

Relying party

owned by root

The role that signs browser users in through authorization-code login and holds their token custody.

Resource server

owned by root

The role that validates access tokens against one configured audience and holds no client identity, grants or introspection.

Sealing key ring

owned by root

One current AES-256-GCM key and retired keys permitted for opening older records. Its lifecycle is controlled by the application for durable stores.

Send evidence

owned by root

The distinction between proven no transmission and possible provider receipt. A timeout or connection failure after submission cannot prove no send.

Service client

owned by root

The confidential role that obtains its own access tokens without a login redirect or a user session.

Session

owned by root

An opaque view of confirmed custody containing verified identity, reference and revision. It is neither a token bag nor a business permission grant.

Session reference

owned by root

A random bearer value naming custody. A reference proves neither that custody still exists nor that the caller may perform a business operation.

Signed assertion

owned by root

An opaque signed JWT credential containing caller-configured claims. Its signature does not make its claims Hellhound-verified identity or access claims.

Store version

owned by root

The compare-and-set concurrency value of a stored row. It is distinct from the token revision and advances on record writes such as refresh reservation or idle touch.

Token revision

owned by root

The generation number of the material exposed by a session. It advances when refreshed material is published.

Tombstone

owned by root

A bounded terminal record preserving consumption or termination evidence after secret material is removed. It prevents replay and late installation recovery within its retention horizon.

Validity deadline

owned by root

The wall-clock instant before which a guarded write may replace a record. It is independent of the record's retention expiry and of the acknowledgement time.

Verified identity

owned by root

The checked identity evidence obtained from an ID token for a consumed login transaction. Its stable key is the pair of issuer and subject.

Write condition

owned by root

The atomic admission requirement for a record write: key absence, an expected store version, or that version before a separate validity deadline.

Write outcome

owned by root

The confirmed result of a conditional record write: Written, VersionConflict or DeadlineElapsed. An unconfirmed result is a store error, never a write outcome.